

Controle de Versão com Git

O que é Git e por que usar?

NobleProg

Git é um sistema de controle de versão distribuído que registra o histórico de alterações de arquivos ao longo do tempo.

Histórico

Cada alteração é gravada com data, autor e mensagem.

Colaboração

Múltiplos devs trabalham em paralelo sem conflito.

Segurança

Nada é perdido — qualquer versão pode ser restaurada.

Branches

Isola novas features do código estável (main).

Instalação & Configuração Inicial

1. Instale o Git: <https://git-scm.com/downloads>

```
# Identifique-se (feito uma só vez)
git config --global user.name "Seu Nome"
git config --global user.email "voce@email.com"

# Verifique a configuração
git config --list
```

2. Criando seu primeiro repositório

```
mkdir meu-projeto
cd meu-projeto
git init          # Inicializa repositório local
git status        # Mostra o estado atual da área de trabalho
```

Fluxo Fundamental: add → commit → push

```
# 1. Ver o que mudou
git status

# 2. Adicionar arquivo(s) à área de stage
git add meuArquivo.sd      # arquivo específico
git add .                  # todos os arquivos alterados

# 3. Registrar o snapshot com mensagem
git commit -m "feat: adiciona cenário CT_01 para rotina X"

# 4. Enviar para o repositório remoto
git push origin main
```

Working
Directory



Stage
(add)



Repositório
Local (commit)



Repositório
Remoto (push)

Branches: Trabalhando em Paralelo

```
# Listar branches
git branch

# Criar e já ir para a branch nova
git checkout -b feature/CT-15-rotina-1122

# Voltar para a branch principal
git checkout main

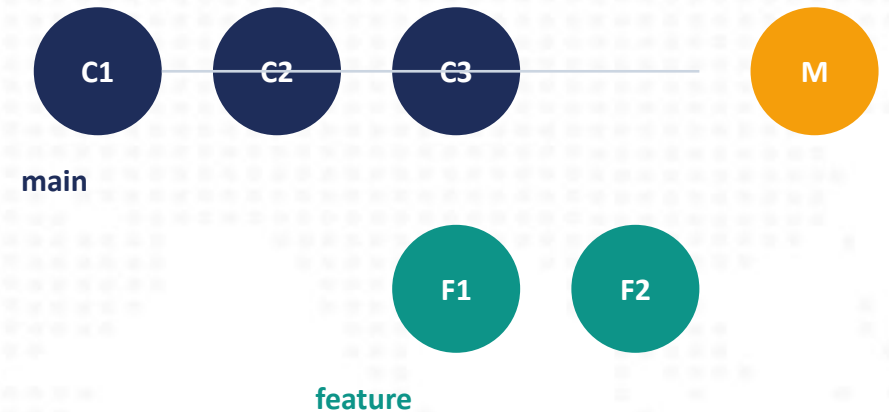
# Incorporar as alterações da feature ao main
git merge feature/CT-15-rotina-1122

# Deletar a branch após merge
git branch -d feature/CT-15-rotina-1122
```

Comandos úteis extras

```
git pull          # Baixar atualizações do remoto
git log --oneline  # Ver histórico resumido
git diff          # Ver diferenças não commitadas
git stash         # Guardar mudanças temporariamente
```

Diagrama



Boas Práticas — Mensagens de Commit

Siga o padrão Conventional Commits para manter o histórico legível:

```
feat: adiciona validação de campo CUSTOFIN  
fix: corrige separador decimal Oracle (vírgula → ponto)  
test: cria CT_03 para cancelamento de pedido  
docs: atualiza README com instrução de ODBC
```

✓ **Faça** Commits pequenos e frequentes; mensagem descritiva no imperativo.

✓ **Faça** Crie uma branch por funcionalidade ou caso de teste.

✗ **Evite** Commitar código quebrado na branch main.

✗ **Evite** Mensagens vazias ou genéricas como 'ajustes' ou 'teste'.